SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence **COURSE CODE:** CSA17

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.*(BL3)*

Assessment Tool Weightage: 35%

QUESTIONS: 3

Duration : 1 Hour
Total Marks:30

Annexure A

Constraint-Based Problem Solving

90/100

Code of Conduct:

I A. Jegadheshwari (192412082) *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

A Jegadheshwari

Annexure A

Constraint-Based Problem Solving

1. A hospital needs to assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night) such that:

   Constraints:

   i. D1 cannot work Night shift
   ii. D2 must work before D3 (Morning < Afternoon < Night)
   iii. Only one doctor per shift
   iv. D3 cannot work Morning
   v. All doctors must be assigned exactly one shift

   Apply Backtracking Search to find a valid assignment with all intermediate steps and constraint checks. State the final valid schedule

2. A robot operates in a grid environment (5×5). It must move from Start (S) to Goal (G). Obstacles block certain paths.

   Constraints:

   i. Movement allowed: Up, Down, Left, Right
   ii. Cost of each move = 1
   iii. Cannot pass through obstacles
   iv. Use Manhattan Distance heuristic

   Using above constraints and BFS Search Algorithm Show step-by-step node expansion and priority queue updates and determine the optimal path and cost.

3. An autonomous rescue robot is deployed in a disaster-affected building to locate and rescue survivors. The robot operates in a partially observable environment where some paths are blocked, and it must make decisions with limited information.

The building is represented as a grid of rooms. The robot starts at position S and must reach a survivor located at G.

Constraints:

- The robot can move up, down, left, right
- Some cells are blocked (obstacles) and cannot be traversed
- The robot has limited sensing capability (can only observe adjacent cells)
- Each movement has a cost of 1, but entering risky zones has an additional cost of 2
- The robot must minimize total cost while ensuring safe navigation
- The robot must update its knowledge dynamically (online search scenario)

Tasks:

a) Analyze all the given constraints and explain how they affect the robot's decision-making process.

b) Develop a solution approach using an appropriate search strategy (e.g., Uniform Cost Search / Online Search Agent). Clearly explain the steps involved.
c) Demonstrate how the robot applies AI concepts such as:

- Intelligent agents
- Rationality
- Environment type

d) Justify why your chosen approach is the most suitable for solving this problem under the given constraints.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

1. A hospital needs to assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night) such that:

Constraints:

i. D1 cannot work Night shift

ii. D2 must work before D3 (Morning < Afternoon < Night)

iii. Only one doctor per shift

iv. D3 cannot work Morning

v. All doctors must be assigned exactly one shift

Apply Backtracking Search to find a valid assignment with all intermediate steps and constraint checks. State the final valid schedule

Application of Backtracking Search for Doctor Shift Scheduling

Backtracking Search is a search algorithm used to solve constraint satisfaction problems (CSPs). It assigns values to variables one by one and checks whether each assignment satisfies the given constraints. If an assignment violates a constraint, the algorithm goes back to the previous step and tries another possible value.

In this problem, a hospital needs to assign three doctors, D1, D2, and D3, to three shifts: Morning, Afternoon, and Night.

The variables are \(D1, D2, D3\), and the possible values for each variable are . Each doctor must be assigned exactly one shift, and each shift must have exactly one doctor.

The given constraints are:

1. D1 cannot work the Night shift.
2. D2 must work before D3, following the order Morning < Afternoon < Night.
3. Only one doctor can be assigned to each shift.
4. D3 cannot work the Morning shift.
5. All doctors must be assigned exactly one shift.

Initially, the schedule is empty. The algorithm starts by selecting D1 and assigning the Morning shift. This assignment satisfies the first constraint because D1 is not assigned to Night. It also satisfies the requirement that only one doctor occupies the Morning shift. Therefore, the assignment D1 = Morning is accepted.

Next, the algorithm selects D2. Since Morning is already occupied by D1, the algorithm tries Afternoon. The assignment D2 = Afternoon satisfies the constraint that only one doctor can occupy a shift. The ordering constraint between D2 and D3 must also be considered. Since D2 must work before D3, D3 must be assigned a shift later than Afternoon. Therefore, D3 can only be assigned Night.

Next, the algorithm selects D3. First, it tries Morning. This assignment is rejected because D3 is not allowed to work the Morning shift, and Morning is already occupied by D1. The algorithm backtracks and tries Afternoon. This assignment is also rejected because Afternoon is already occupied by D2. The algorithm backtracks again and tries Night. Night is available, and assigning D3 to Night satisfies the constraint that D2 must work before D3. It also satisfies the constraint that D3 cannot work Morning.

The final schedule is shown below.

Doctor	Assigned shift
D1	Morning
D2	Afternoon
D3	Night

The constraint verification is as follows:

Constraint	Verification	Status
D1 cannot work Night	D1 is assigned Morning	Satisfied
D2 must work before D3	Afternoon comes before Night	Satisfied
Only one doctor per shift	Each shift has one doctor	Satisfied
D3 cannot work Morning	D3 is assigned Night	Satisfied
Every doctor must receive exactly one shift	All three doctors are assigned once	Satisfied

Thus, Backtracking Search successfully finds a valid schedule for the hospital. The final assignment is D1 – Morning, D2 – Afternoon, and D3 – Night. All five constraints are satisfied, and the hospital can use this schedule to assign the doctors to their respective shifts.

2. A robot operates in a grid environment (5×5). It must move from Start (S) to Goal (G).

Obstacles block certain paths.

Constraints:

i. Movement allowed: Up, Down, Left, Right

ii. Cost of each move = 1

iii. Cannot pass through obstacles

iv. Use Manhattan Distance heuristic

Using above constraints and BFS Search Algorithm Show step-by-step node expansion

and priority queue updates and determine the optimal path and cost.

2. Application of BFS Search Algorithm for Robot Pathfinding in a 5×5 Grid

Breadth-First Search (BFS) is an uninformed search algorithm that explores nodes level by level. It uses a FIFO (First In, First Out) queue to store the nodes waiting to be explored. Since the cost of every movement is 1, BFS finds an optimal path with the minimum number of moves when obstacles are avoided.

Consider a 5×5 grid in which a robot must move from the Start (S) to the Goal (G). The robot can move Up, Down, Left, or Right. It cannot move through obstacles, and each movement has a cost of 1.

Since the exact obstacle positions are not specified in the question, the following example assumes the grid shown below.

Grid representation

Start

Goal

Obstacle

Free cell

The movement cost is 1, and the Manhattan Distance heuristic is

$$ h(n)=|x_n-x_G|+|y_n-y_G| $$

where (x_n,y_n) is the current cell and (x_G,y_G) is the goal cell.

Note: BFS itself does not use a heuristic or a priority queue. The Manhattan Distance heuristic is normally used with Greedy Best-First Search or A*. For BFS, a FIFO queue is used. The steps below demonstrate BFS with queue updates and calculate Manhattan distances for reference.

Step-by-step node expansion

Use coordinates (r,c), starting at $(1,1)$ and ending at $(5,5)$. Neighbours are considered in the order Right, Down, Left, Up. Obstacles and previously visited cells are skipped.

Step	Expanded node	Valid new neighbours	Queue after expansion
1	S (1,1)	(1,2), (2,1)	[(1,2), (2,1)]
2	(1,2)	(2,2)	[(2,1), (2,2)]
3	(2,1)	(3,1)	[(2,2), (3,1)]
4	(2,2)	(3,2)	[(3,1), (3,2)]
5	(3,1)	None; adjacent cells blocked or visited	[(3,2)]
6	(3,2)	(3,3)	[(3,3)]
7	(3,3)	(3,4), (4,3)	[(3,4), (4,3)]

8	(3,4)	(2,4), (3,5)	[(4,3), (2,4), (3,5)]
9	(4,3)	None; adjacent cells blocked or visited	[(2,4), (3,5)]
10	(2,4)	(1,4), (2,5)	[(3,5), (1,4), (2,5)]
11	(3,5)	(4,5)	[(1,4), (2,5), (4,5)]
12	(1,4)	(1,5)	[(2,5), (4,5), (1,5)]
13	(2,5)	None; adjacent cells blocked or visited	[(4,5), (1,5)]
14	(4,5)	G (5,5)	[(1,5), G]
15	(1,5)	None; adjacent cells blocked or visited	[G]
16	G (5,5)	Goal reached	[]

Optimal path

Tracing the parent nodes from the goal back to the start gives the path:

$$\begin{aligned} S&\rightarrow(1,2)\rightarrow(2,2)\\ &\rightarrow(3,2)\rightarrow(3,3)\\ &\rightarrow(3,4)\rightarrow(3,5)\\ &\rightarrow(4,5)\rightarrow G \end{aligned}$$

The total path cost is

$$\text{Cost}=8\times1=\boxed{8}$$

Result

Thus, BFS explores the grid level by level using a FIFO queue and avoids obstacles. For the assumed grid, the robot reaches the goal using an optimal path of 8 moves with a total cost of 8. The Manhattan Distance heuristic can be calculated at each node, but it does not determine BFS expansion order.

3. An autonomous rescue robot is deployed in a disaster-affected building to locate and rescue survivors. The robot operates in a partially observable environment where some paths are blocked, and it must make decisions with limited information.

The building is represented as a grid of rooms. The robot starts at position S and must reach a survivor located at G.

Constraints:

- The robot can move up, down, left, right
- Some cells are blocked (obstacles) and cannot be traversed
- The robot has limited sensing capability (can only observe adjacent cells)
- Each movement has a cost of 1, but entering risky zones has an additional cost of 2
- The robot must minimize total cost while ensuring safe navigation

 The robot must update its knowledge dynamically (online search scenario)

Tasks:

a) Analyze all the given constraints and explain how they affect the robot’s decision-making process.

b) Develop a solution approach using an appropriate search strategy (e.g., Uniform Cost Search / Online Search Agent). Clearly explain the steps involved.

c) Demonstrate how the robot applies AI concepts such as:

 Intelligent agents

 Rationality

 Environment type

d) Justify why your chosen approach is the most suitable for solving this problem under the given constraints.

3. Autonomous Rescue Robot Navigation Using Online Search and Uniform Cost Search

a) Analysis of the Given Constraints

An autonomous rescue robot is deployed in a disaster-affected building to locate a survivor. The building is represented as a grid of rooms. The robot starts at position S and must reach the goal position G while avoiding obstacles and minimizing the total movement cost.

The given constraints affect the robot's decision-making process as follows.

1. Movement constraints: The robot can move only Up, Down, Left, and Right. It cannot move diagonally. Therefore, it must select one of the four permitted directions at each step.

2. Obstacles: Some cells are blocked due to damaged walls, debris, or other hazards. The robot must identify blocked cells through sensing and avoid them while selecting a path.

3. Limited sensing capability: The robot can observe only adjacent cells. It does not have complete information about the entire building. Therefore, it must explore the environment, collect information, and update its internal map dynamically.

4. Movement cost: Each movement has a basic cost of 1. Entering a risky zone adds an additional cost of 2. Therefore, entering a risky cell costs \(1+2=3\). The robot must consider these costs while selecting a route.

5. Minimum-cost and safe navigation: The robot must reach the survivor while avoiding blocked cells and preferring safer, lower-cost routes. It should not select a shorter path if its total cost is greater than that of a safer alternative.

6. Dynamic knowledge update: The robot must update its map whenever it discovers a new obstacle, safe cell, or risky zone. If the selected path becomes blocked or more expensive, it must reconsider its route.

These constraints make the problem a partially observable pathfinding problem in which the robot must continuously sense, plan, act, and update its knowledge.

b) Solution Approach Using Uniform Cost Search and an Online Search Agent

Uniform Cost Search (UCS) is suitable for finding a minimum-cost path when movement costs are non-negative. Since the robot has limited information about the environment, it can use an online search agent with UCS-based replanning. The robot applies UCS to the currently known map and replans whenever new information changes the available paths or their costs.

The cost function is

$$ c(n,n')= \begin{cases} \infty, & \text{if the next cell is blocked}\\ 3, & \text{if the next cell is risky}\\ 1, & \text{if the next cell is safe} \end{cases} $$

The cumulative path cost is

$$ g(n)=\sum_{i=1}^{n}c_i $$

The robot selects the reachable path with the smallest cumulative cost.

Steps involved:

1. Initialization: Place the robot at the starting position S. Initialize its internal map with the starting cell and any available information. Set the initial path cost to zero.

2. Sense the surroundings: Observe the adjacent cells in the Up, Down, Left, and Right directions. Identify known obstacles, safe cells, and risky cells.

3. Update the map: Record the newly observed information in the robot's internal map. Unknown cells remain unclassified until the robot senses them.

4. Generate possible movements: Consider all accessible neighbouring cells. Reject any cell known to be blocked.

5. Calculate movement costs: Assign a cost of 1 to a safe cell and a cost of 3 to a risky cell.

6. Apply Uniform Cost Search: Maintain a priority queue ordered by cumulative path cost $(g(n))$. Expand the reachable node with the lowest cumulative cost first.

7. Select the next action: Choose the next movement based on the lowest-cost feasible route in the currently known map, while accounting for safety requirements.

8. Move and observe: Execute one movement, sense the adjacent cells again, and update the map.

9. Replan when necessary: If a newly discovered obstacle blocks the planned route, or new information changes the route costs, update the search and select another feasible route.

10. Terminate: Continue the sense–plan–act cycle until the robot reaches G. Report the total cost of the path actually travelled.

For example, suppose the robot discovers two routes:

- Route A: 5 safe movements and 2 risky movements.
- Route B: 8 safe movements and no risky movements.

The cost of Route A is

$$ C_A=(5\times1)+(2\times3)=11 $$

The cost of Route B is

$$ C_B=8\times1=8 $$

Since $8<11$, the robot selects Route B because it has the lower total cost.

This example assumes both routes are feasible and their cell types are known. In the actual partially observable environment, the robot may need to revise its decision as it discovers new information.

c) Application of Artificial Intelligence Concepts

1. Intelligent Agents

An intelligent agent perceives its environment through sensors and acts on the environment using actuators. In this problem, the rescue robot acts as an intelligent agent.

- Sensors: Detect adjacent obstacles, safe cells, and risky zones.
- Internal knowledge: Stores the robot's current position and its discovered map.
- Decision-making: Uses search algorithms to select a suitable movement.
- Actuators: Move the robot Up, Down, Left, or Right.
- Goal: Reach the survivor while minimizing travel cost and maintaining safety.

The robot repeatedly senses, updates its knowledge, selects an action, and executes that action. This is an example of an intelligent agent operating in a changing environment.

2. Rationality

A rational agent selects actions that best achieve its goal based on the information currently available. The rescue robot must consider movement costs, known obstacles, risky zones, and the possibility of discovering new hazards.

For example, if one known route costs 11 units and another feasible route costs 8 units, the robot chooses the route costing 8 units. If new information reveals an obstacle, the rational action is to update the map and replan instead of continuing toward the blocked cell.

Rationality does not mean that the robot always knows the globally optimal route in advance. It means that its decisions are appropriate given its available information, cost model, and safety requirements.

3. Environment Type

The rescue robot operates in the following types of AI environments:

Environment property	Classification	Explanation
Observability	Partially observable	The robot observes only adjacent cells.
Determinism	Assumed deterministic	Each permitted movement leads to the selected neighbouring cell, unless the model is extended to include movement uncertainty.
Episodic or sequential	Sequential	Each movement affects the robot's later decisions.
Static or dynamic	Dynamic	Obstacles or risk conditions may change during rescue operations.
State space	Discrete	The robot moves between individual grid cells.
Number of agents	Single-agent	The robot makes its own navigation decisions.
Knowledge	Initially incomplete	The robot discovers the environment through sensing.
Cost structure	Non-uniform	Safe movements cost 1 and risky movements cost 3.

These properties explain why the robot requires continuous observation, planning, and knowledge updates rather than following a fixed path from the beginning.

d) Justification for the Chosen Approach

The combination of an online search agent and Uniform Cost Search is appropriate because the robot must navigate with incomplete information and minimize a non-uniform movement cost.

First, UCS considers cumulative path cost rather than simply counting the number of movements. This is important because entering a risky zone costs 3 units, whereas entering a safe cell costs 1 unit. The lowest-cost route may therefore be longer in terms of the number of cells travelled.

Second, the online search agent supports decision-making with limited sensing. The robot observes adjacent cells, updates its internal map, and uses the latest information to select its next action. This allows it to respond to newly discovered obstacles and risky zones.

Third, the approach supports replanning. When the robot discovers that a previously considered route is blocked or more expensive than expected, it can update the known graph and calculate a new route.

Fourth, the approach is systematic and suitable for grid-based navigation. UCS uses a priority queue to expand the lowest-cost known frontier node first. With non-negative movement costs and a fixed, fully known graph, UCS guarantees a minimum-cost path to the goal. In a partially observable environment, online replanning improves navigation, although the robot cannot guarantee the globally optimal route before discovering the relevant parts of the building.

Finally, safety requirements must be incorporated into the planning model. Blocked cells must never be traversed, and risky cells should be penalized or avoided when an acceptable safer route exists. If a risky zone is considered too dangerous to enter, it should be treated as forbidden rather than merely assigned a higher cost.

Thus, an online search agent using UCS-based replanning provides a suitable solution for autonomous rescue navigation because it accounts for non-uniform movement costs, limited sensing, dynamic knowledge updates, obstacle avoidance, and minimum-cost path selection.

Result: The rescue robot can navigate the disaster-affected building by repeatedly sensing adjacent cells, updating its internal map, calculating path costs, and replanning as necessary. It aims to reach the survivor through a safe, feasible route with the minimum total cost supported by its available information.